

PrivacyGuard

Multilingual PII Detection and Redaction

A Comparative NLP Study of Rule-Based, Classical Machine Learning, and Deep Learning Approaches for Detecting and Redacting Personally Identifiable Information in English, Bengali, and Banglish Text

Project Report

Natural Language Processing Laboratory — Level 4, Term 1
Department of Computer Science and Engineering

Abstract

PrivacyGuard is a multilingual natural language processing system designed to detect and redact personally identifiable information (PII) in English, Bengali, and Banglish (code-mixed Bengali-English) text. The project addresses two complementary NLP tasks: document-level PII classification and fine-grained named-entity recognition (NER) that locates the exact character span of each sensitive entity.

Two families of models were implemented and evaluated. For document classification, three classical algorithms — Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machines — were trained over TF-IDF features, each with both a scikit-learn implementation and a fully hand-written (“from scratch”) NumPy implementation of the underlying mathematics. For entity-level detection, a bidirectional LSTM (BiLSTM) sequence labeler and a fine-tuned multilingual BERT (mBERT) token classifier were trained on a synthetically generated, BIO-tagged dataset covering eighteen PII entity types. A deterministic rule/pattern detector provides a zero-training baseline and a safety-net fallback.

All models feed into a shared inference pipeline that assigns a confidence score, maps each detected entity to a four-level risk category (LOW to CRITICAL), and performs span-safe, reverse-offset text redaction so that overlapping replacements never corrupt the original character offsets. The system is exposed through an interactive Streamlit dashboard offering nine selectable models and a FastAPI pattern-scanning microservice. On the held-out synthetic test data, all classical models reached 1.000 weighted F1, the BiLSTM reached 1.000 validation entity-token F1, and mBERT reached 0.9999 entity-token F1 — results that reflect strong learning of the generated template distribution and are explicitly framed as not yet validated against real-world, human-annotated text.

1. Introduction

Digital communication, help-desk tickets, medical intake forms, chat logs, and student records frequently contain personally identifiable information — names, phone numbers, national ID numbers, email addresses, and health conditions — mixed freely with ordinary prose. In Bangladesh and similar multilingual settings, this text is rarely monolingual: it blends formal Bengali, colloquial Banglish (Bengali written with Latin characters), and English within a single sentence. Manually scrubbing such text before storage, sharing, or model training is slow, inconsistent, and does not scale.

PrivacyGuard was built to automate this task end to end: given raw text, the system must locate every PII span, classify its type, score its privacy risk, and produce a redacted version of the text that is safe to store, log, or forward — while remaining usable, interpretable, and fully local (no text leaves the machine and no third-party generative API is called).

1.1 Problem Statement

Given an arbitrary input sentence or document in English, Bengali, or Banglish, automatically:

1. determine whether the text contains PII at all (document-level classification);
2. locate the exact character span, entity type, and confidence of every PII mention (token-level sequence labeling);
3. assign each mention a privacy risk level; and
4. produce a redacted, human-readable version of the text with each PII span replaced by a type-specific marker, without corrupting the surrounding text.

1.2 Objectives

- **Comparative modeling:** implement and fairly compare a rule-based detector, three classical machine-learning classifiers (in both library and from-scratch form), and two neural sequence labelers on the same task family.
- **Multilingual coverage:** support English, Bengali, and Banglish within one unified entity schema.
- **Exact-offset redaction:** guarantee that redaction never shifts or corrupts unrelated text, even with multiple overlapping or adjacent entities.
- **Usable interfaces:** expose every model through an interactive dashboard and a lightweight API so the system can be demonstrated and evaluated end to end.
- **Reproducibility:** seed every random process (data generation, splitting, and training) so that results and datasets can be regenerated deterministically.

2. What the System Does

The system accepts free-form text and returns a structured list of detected entities plus a redacted version of the input. For example:

Input

My name is Rahim Ahmed. Email me at rahim@example.com or call 01712345678.

Structured entity output (one record per detected entity)

```
{
  "entity": "rahim@example.com",
  "type": "EMAIL",
  "start": 36, "end": 53,
  "confidence": 0.99,
  "source": "pattern",
  "risk": "HIGH", "score": 3
}
```

Redacted output

My name is [PERSON]. Email me at [EMAIL] or call [PHONE_NUMBER].

This structured output — exact offsets, entity type, confidence, source model, and risk — is what allows the redaction step to be span-safe and the risk-scoring step to be traceable back to specific text.

2.1 Two NLP Tasks, Nine Selectable Models

The project deliberately frames PII detection as two distinct NLP tasks rather than one, because they require different model families and produce different guarantees:

Task	Models	Output	Exact spans?
Entity detection / NER	Pattern detector, BiLSTM, BERT	Entity type, offsets, confidence	Yes
Document classification	Logistic Regression, Naive Bayes, SVM (+ scratch variants)	Whole text labeled PII or SAFE	No — patterns supply spans

The Streamlit interface exposes all nine as independently selectable options: Pattern detector, BERT, BiLSTM, Logistic Regression, Logistic Regression (Scratch), Naive Bayes, Naive Bayes (Scratch), SVM, and SVM (Scratch). If a chosen model's saved artifact is missing, incompatible, or fails to load, the system can optionally fall back to the pattern detector so the demo never hard-fails.

3. Complete System Architecture

The diagram below shows the full system in two halves: the training-time pipeline that produces every saved model artifact (top), and the inference-time pipeline that turns raw input text into a redacted, risk-scored result (bottom).

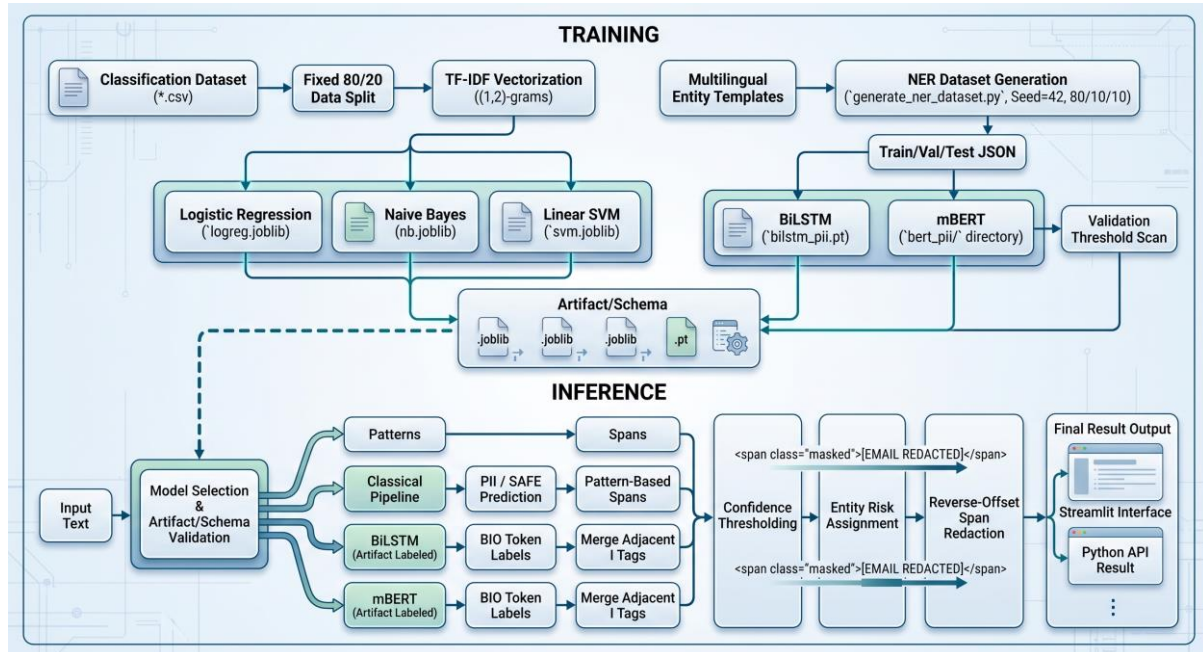


Figure 1. PrivacyGuard end-to-end architecture — training pipeline (top) and inference pipeline (bottom).

At training time, the three document classifiers learn from shared TF-IDF features, while BiLSTM and mBERT independently learn BIO token labels from the generated NER dataset. At inference time, the model registry validates that a requested model's saved artifact and label schema are compatible before loading it; the selected model then returns either a document-level prediction or a set of entity spans. Confidence filtering, risk assignment, and reverse-offset redaction are shared, model-agnostic post-processing steps applied uniformly regardless of which model produced the entities.

3.1 Design Rationale

- **Model registry with compatibility checks:** BiLSTM and BERT artifacts store their label schema alongside the weights; the registry refuses to load a checkpoint whose label order does not match the current 28-label schema, preventing silently incorrect predictions from a stale checkpoint.
- **Shared post-processing:** confidence thresholding, risk scoring, and redaction are implemented once and reused by every model, so the nine model choices are fairly comparable at the output stage.
- **Fail-safe fallback:** if a learned model cannot be loaded or throws during inference, the system can transparently fall back to the zero-training pattern detector rather than returning an error to the user.

4. Datasets

The two NLP tasks use two different datasets because they require different label formats.

4.1 Document-Classification Dataset

Located at `data/processed/classification_dataset.csv`, this dataset has three columns: `id`, `text`, and a binary label (1 = PII, 0 = SAFE). The current file contains 6,000 records — 3,730 labeled PII and 2,270 labeled SAFE. Training performs a stratified 80/20 train-test split with a fixed `random_state=42`, and TF-IDF vectorization is fitted only on the training split (inside a scikit-learn Pipeline) to avoid leaking test vocabulary into the features.

4.2 Multilingual NER Dataset

The neural sequence labelers (BiLSTM and mBERT) are trained on a synthetically generated, BIO-tagged dataset produced by `src/dataset/generate_ner_dataset.py`. Generation is fully reproducible given the same sample count and seed:

```
python -m src.dataset.generate_ner_dataset --samples 30000 --seed 42
```

The generator (1) synthesizes realistic entity values for all eighteen PII types, (2) fills English, Bengali, Banglish, long-form, and PII-free negative templates, (3) assigns BIO labels directly at insertion time so labels are always correct by construction, (4) shuffles records with a locally seeded generator, and (5) writes an 80/10/10 train/validation/test split.

File	Records (current)	Purpose
<code>ner_dataset.json</code>	30,000	Complete generated dataset
<code>train.json</code>	24,000	BiLSTM / BERT model fitting
<code>validation.json</code>	3,000	Checkpoint selection and threshold scan
<code>test.json</code>	3,000	Held-out neural-model evaluation

Each record is a simple token/label pair, for example:

```
{
  "tokens": ["Rahim", "Ahmed", "lives", "in", "Dhaka", "."],
  "labels": ["B-PERSON", "I-PERSON", "O", "O", "B-LOCATION", "O"]
}
```

5. Supported Entities, BIO Labels, and Risk Levels

The shared neural label schema (`src/models/transformer/labels.py`) covers eighteen PII entity types, each assigned a fixed privacy-risk tier used later for redaction prioritization and document-level risk summaries.

Entity	Example	Risk
PERSON	Rahim Ahmed	MEDIUM
LOCATION	Dhaka	LOW
ORGANIZATION	BRAC Bank	MEDIUM
ADDRESS	House 25 Road 7 Dhanmondi	HIGH
PHONE	01712345678	HIGH
EMAIL	user@example.com	HIGH
NID	1234567890	CRITICAL
CREDIT_CARD	4532 0151 1283 0366	CRITICAL
ACCOUNT	AC0123456789	CRITICAL
EMPLOYEE_ID	EMP-45892	HIGH
MEDICAL_ID	MED-32145	CRITICAL
STUDENT_ID	CSE-2024-1025	HIGH
IP_ADDRESS	192.168.10.5	HIGH
DATE	21 February 2001	MEDIUM
PASSPORT	A1234567	CRITICAL
HEALTH_CONDITION	high blood pressure	CRITICAL
OCCUPATION	software engineer	MEDIUM
EDUCATION	Computer Science	MEDIUM

5.1 BIO Tagging Scheme

- **B-TYPE** — first token of an entity mention.
- **I-TYPE** — a continuation token inside a multi-token entity.
- **O** — a token outside any entity.
- **-100** — a training-only ignore value used for padding positions and repeated BERT sub-word pieces.

Nine of the eighteen types are multi-token and therefore support an I- continuation label: PERSON, LOCATION, ORGANIZATION, ADDRESS, CREDIT_CARD, DATE, HEALTH_CONDITION, OCCUPATION, and

EDUCATION. Counting one O label, eighteen B- labels, and nine I- labels gives a complete schema of 28 labels — the fixed output dimensionality of both the BiLSTM and BERT classification heads.

5.2 Risk Scoring

Score	Level	Example entity types
1	LOW	Location
2	MEDIUM	Person, date, organization, occupation, education
3	HIGH	Phone, email, IP address, address, employee / student ID
4	CRITICAL	National ID, credit card, account, passport, medical ID, health condition

The overall risk reported for a document is the maximum entity-level score found in it, not a sum — a single CRITICAL entity makes the whole document CRITICAL regardless of how many LOW-risk entities also appear.

6. Preprocessing and Text Representation

A single preprocessing pipeline is not shared across all models, because each architecture needs a different balance between text normalization and exact character/offset preservation.

6.1 Runtime Span Tokenization

Every neural and pattern-based path first runs a lightweight span tokenizer (`_lexical_spans()` in `src/inference/model_service.py`) that finds non-whitespace tokens, separates edge punctuation, and — critically — preserves the exact start/end character offsets of every token, while keeping structured values such as emails, phone numbers, EMP-45892, and CSE-2024-1025 intact as single tokens. These offsets are what allow the redaction stage to slice the original text precisely.

6.2 Classical TF-IDF Representation

The document classifiers use a scikit-learn `TfidfVectorizer` configured to lowercase input, use the default word token pattern with no stop-word removal, no stemming, and no lemmatization, extract word unigrams and bigrams (`ngram_range=(1,2)`), and cap the vocabulary at 10,000 features. For the phrase “my phone number”, this produces unigrams `my`, `phone`, `number` and bigrams `my phone`, `phone number`. TF-IDF weighting emphasizes terms that are frequent within one document but comparatively rare across the whole collection. This is a bag-of-n-grams feature representation for classification — not an n-gram language model used for next-word prediction or generation.

6.3 Legacy Preprocessing Module

`src/preprocessing/` implements a fuller NLP pipeline — Unicode NFKC normalization, repeated-character reduction, HTML/URL cleaning, lowercasing, NLTK tokenization, and informal spelling normalization — but it is not wired into the active training pipeline. It is documented here as available infrastructure rather than an active step, since its ASCII-only cleaner would strip Bengali text if applied unchanged to the multilingual NER data.

7. Model Pipelines, End to End

This section walks through every model the system supports, from raw input to final prediction, in the order a reader would naturally compare them: no training, classical training, and neural training.

7.1 Pattern and Context Detector (Zero-Training Baseline)

Implemented in `src/redaction/entity_detector.py`, this is a deterministic, rule-based detector that requires no training data at all:

```
raw text
-> structured regex candidates (email, BD phone, NID, IP, card candidate)
-> Luhn checksum validation for credit-card candidates
-> contextual PERSON candidates (name-cue patterns)
-> contextual / gazetteer LOCATION candidates
-> overlap resolution: pattern > context > gazetteer > full phrase
-> ordered, non-overlapping exact spans
-> fixed, hand-assigned confidence and risk
-> thresholded redaction
```

It supports Bengali digits for numeric validation, runs with very low latency since no model forward pass is needed, and returns exact offsets by construction. Its coverage is narrower than the full 18-type neural schema, but it is the system's safety-net fallback: any learned model that fails to load or throws at inference time can transparently degrade to this detector so that a demo, a help-desk pipeline, or an API caller still receives a usable result.

7.2 Shared Classical Document-Classification Pipeline

Logistic Regression, Multinomial Naive Bayes, and Linear SVM all sit on top of one identical data and feature pipeline, which is what makes their results directly comparable:

```
classification_dataset.csv
-> drop rows with missing text/label
-> stratified 80/20 split, seed=42
-> TF-IDF: word (1,2)-grams, max 10,000 features
-> classifier.fit(X_train, y_train)
-> predict document label: PII / SAFE
-> weighted accuracy, precision, recall, F1
-> save fitted pipeline as a .joblib artifact
```

At inference time, the saved TF-IDF vectorizer and classifier jointly produce a PII/SAFE label for the whole input; because a document classifier cannot itself say which characters triggered the decision, the pattern detector is additionally run to supply the actual redaction spans.

7.2.1 Logistic Regression

```
TF-IDF x -> linear score  $w \cdot x + b$  -> sigmoid -> binary decision -> PII / SAFE
```

Configuration: max_iter=1000, class_weight="balanced" (to compensate for the 3,730 / 2,270 class imbalance). Artifact: models_saved/logistic_regression.joblib.

7.2.2 Multinomial Naive Bayes

```
TF-IDF -> class priors  $P(\text{class})$  -> feature likelihoods  $P(\text{feature} \mid \text{class})$ 
-> compare joint log-likelihood per class -> PII / SAFE
```

A fast probabilistic baseline built on a simplifying conditional-independence assumption between features. Artifact: models_saved/naive_bayes.joblib.

7.2.3 Linear Support Vector Machine

```
TF-IDF -> maximum-margin linear decision boundary -> signed distance -> PII / SAFE
```

Configuration: class_weight="balanced". The current SVM does not expose calibrated class probabilities, only a signed decision score. Artifact: models_saved/svm.joblib.

7.2.4 “From-Scratch” Classical Implementations

To demonstrate the underlying mathematics rather than only calling a library estimator, all three classifiers were re-implemented by hand in NumPy on top of the same TF-IDF features, so the from-scratch and scikit-learn results remain directly comparable:

Scratch model	Manually implemented operations
Logistic Regression	$w \cdot x + b$, sigmoid activation, binary cross-entropy loss, analytical gradients, and batch gradient descent
Multinomial Naive Bayes	class priors, per-feature counts, Laplace (add- α) smoothing, and joint log-likelihood scoring
Linear SVM	hinge loss, L2 regularization, subgradient computation, and iterative parameter updates

Only the classifier mathematics is hand-written; TF-IDF feature extraction itself still comes from scikit-learn so that features are identical across both variants.

7.3 BiLSTM Sequence Labeler

The BiLSTM is the first of the two neural sequence labelers: instead of classifying a whole document, it assigns one of 28 BIO labels to every token, which is what makes exact-span entity detection possible without relying on the rule-based patterns.

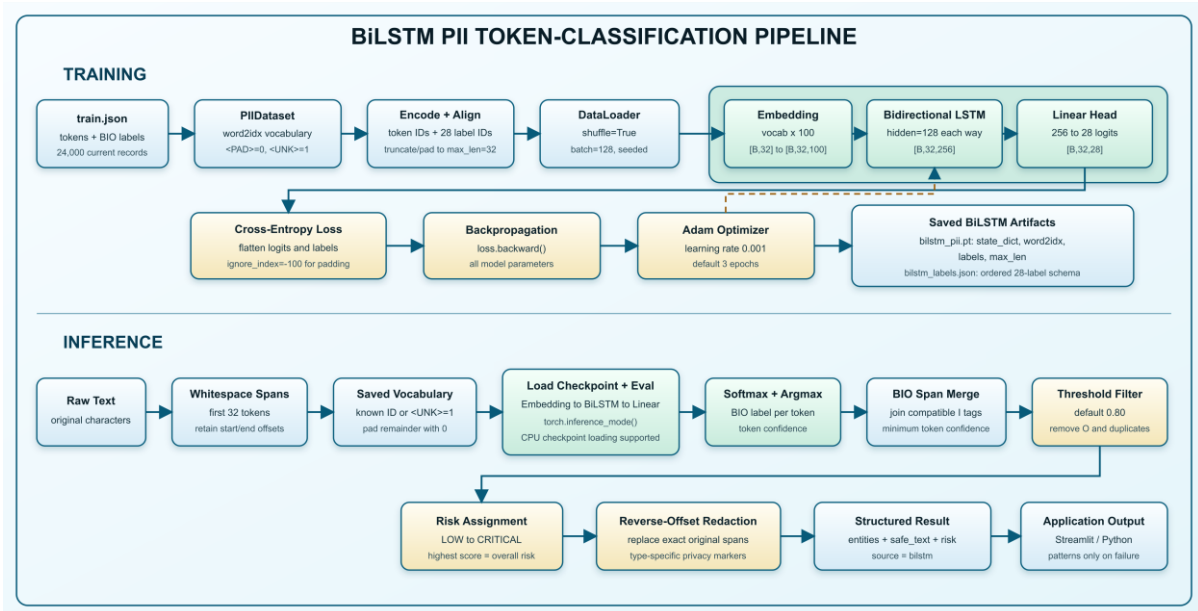


Figure 2. BiLSTM training and inference pipeline.

Training

```
train.json
-> Unicode / case / digit normalization + reusable email shape token
-> minimum-frequency vocabulary (<PAD>=0, <UNK>=1)
-> token IDs, pad/truncate training windows to 128
-> word dropout (trains a useful <UNK> representation)
-> embedding layer: vocabulary_size x 100
-> packed sequences (padding ignored by the RNN)
-> bidirectional LSTM: hidden size 128 per direction
-> concatenate both directions -> 256 features / token
-> dropout
-> linear layer: 256 -> 28 label logits (batch, 128, 28)
-> class-weighted cross-entropy, ignore_index=-100
-> AdamW optimizer + gradient clipping
-> validation threshold scan + entity-token metrics
-> early stopping, save best checkpoint
```

Inference

```
text -> exact lexical spans across the full document
-> overlapping 128-token windows, 32-token overlap
-> normalized token IDs from the saved vocabulary
-> packed BiLSTM forward pass -> softmax over 28 BIO labels
-> keep the prediction with the strongest boundary context
-> merge compatible I- tags into full entity spans
-> validation-selected confidence threshold -> risk -> redaction
```

Padding positions use label -100 rather than O, so `CrossEntropyLoss(ignore_index=-100)` never rewards the model for predicting “outside” at an empty position. The saved checkpoint bundles its own vocabulary, because rebuilding the vocabulary from scratch could assign different token IDs and silently corrupt predictions.

Strengths: bidirectional context, sliding-window inference over documents of any length, a comparatively small and fast model, and training code that is easy to read and reason about. Limitations: genuinely unseen names or misspellings still collapse to a single shared <UNK> vector, and because the training data is synthetic and template-based, the model can be confidently wrong on real-world writing that differs structurally from the templates.

7.4 Multilingual BERT Token Classifier

The second sequence labeler fine-tunes google-bert/bert-base-multilingual-cased, a 12-layer transformer pretrained across 104 languages including Bengali, and adds a token-classification head over the same 28-label BIO schema used by the BiLSTM.

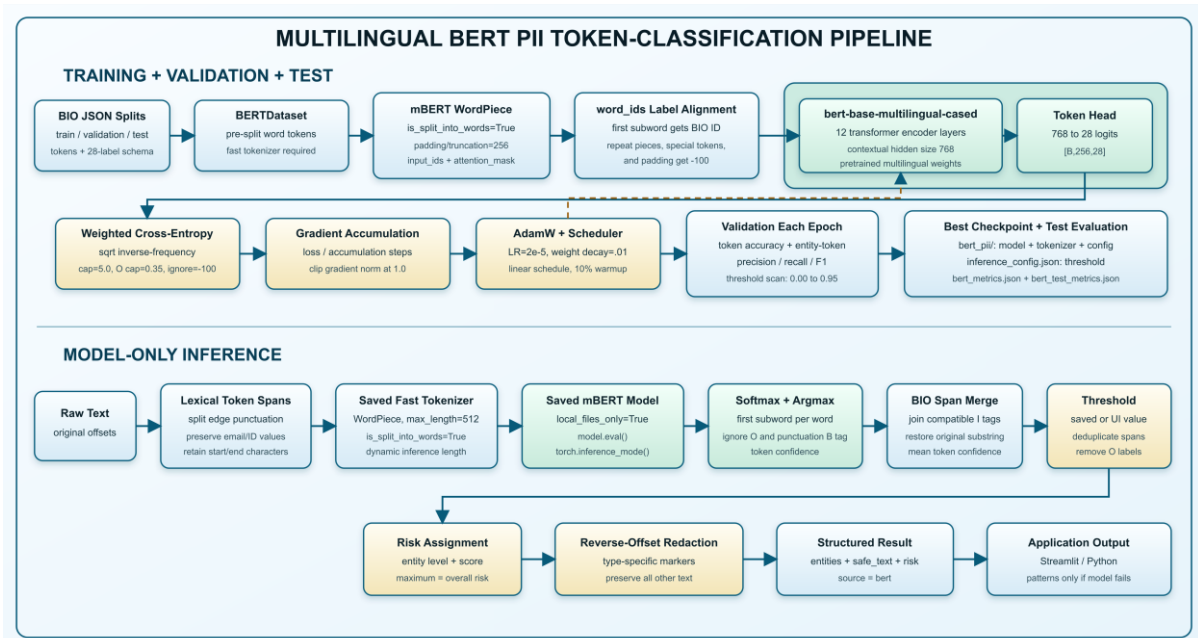


Figure 3. Multilingual BERT training, validation, and inference pipeline.

Training

```
train.json
-> fast mBERT tokenizer, pre-split words (is_split_into_words=True)
-> WordPiece sub-words + word_ids alignment
    first sub-word of a word gets the word's BIO label
    other sub-word pieces, special tokens, and padding get -100
-> pad / truncate to 256 sub-word positions
-> multilingual BERT encoder (12 layers, hidden size 768)
-> token classification head: 768 -> 28 logits
-> weighted cross-entropy
    sqrt inverse-frequency class weights, cap 5.0, 'O' capped at 0.35
-> AdamW, default learning rate 2e-5, weight decay 0.01
```

```
-> linear LR schedule with 10% warm-up, gradient clipping at 1.0
-> validation after every epoch
-> save the checkpoint with the best validation entity-token F1
```

Gradient accumulation gives an effective batch size of $\text{batch_size} \times \text{accumulation_steps}$, which lets larger effective batches run on limited GPU/CPU memory. Validation additionally scans confidence thresholds from 0.0 to 0.95 and stores the best-performing threshold in `models_saved/bert_pii/inference_config.json` for use at inference time.

Inference

```
text -> offset-preserving lexical tokens
      -> saved tokenizer, max 512 sub-word positions
      -> BERT forward pass -> logits -> softmax
      -> take the first sub-word's prediction for each lexical token
      -> merge compatible I- tags into full entity spans
      -> mean confidence across the merged sub-word tokens
      -> saved threshold -> risk -> redaction
```

Strengths: multilingual pretrained contextual representations and much better generalization to unseen words through sub-word tokenization, compared to the BiLSTM's closed vocabulary. Limitations: substantially higher CPU training cost, a larger saved artifact, and performance that still depends on how representative the synthetic training templates are of real text.

7.5 Experimental / Unused Components

Two modules exist in the codebase but are not part of the active nine-model pipeline: `src/features/ngram_features.py` (experimental word 1-to-3-gram and character 3-to-5-gram count utilities) and `src/features/embeddings.py` (a 100-dimensional skip-gram Word2Vec implementation with mean sentence embeddings). Both are documented here for completeness since they represent additional NLP techniques explored during the project, but `train_all.py`, the Streamlit app, and the model registry do not call either of them.

8. Confidence, Risk, and Span-Safe Redaction

8.1 Confidence Is Not Directly Comparable Across Models

- **Pattern detector:** confidence values are manually assigned by rule.
- **BERT / BiLSTM (single token):** confidence is the maximum softmax probability.
- **BERT (merged entity):** confidence is the mean confidence across the merged sub-word tokens.
- **BiLSTM (merged entity):** confidence is the minimum confidence across the merged tokens (a conservative choice).
- **SVM:** no calibrated entity-level probability is available; classical redaction spans instead come from the pattern detector.

Lowering the confidence threshold increases recall (fewer missed entities) at the cost of more false positives; softmax probability is not a guarantee of real-world correctness, and low neural confidence can

stem from limited training data, class imbalance, unfamiliar tokens, ambiguous context, or an incompatible checkpoint.

8.2 Span-Safe Reverse-Offset Redaction

The redactor performs three steps: (1) keep only entities meeting the active confidence threshold; (2) sort the surviving spans from the end of the text toward the start; (3) replace each exact text[start:end] slice with its type-specific marker. Processing spans in reverse order guarantees that replacing one entity never shifts the character offsets of entities still waiting to be replaced — a common bug class in naive left-to-right redaction.

Entity type	Redaction marker
PERSON	[PERSON]
PHONE	[PHONE_NUMBER]
EMAIL	[EMAIL]
ACCOUNT	[ACCOUNT_NUMBER]
Unknown / other	[PRIVATE_DATA]

8.3 Failure Fallback

Calling `analyze(text, model_name, fallback=True)` catches model loading or inference errors and transparently substitutes the pattern detector; the returned result reports both the model that was requested and the model that actually produced the output, so silent degradation is still auditable. Setting `fallback=False` instead surfaces the original exception, which is the recommended mode while debugging a specific model.

9. Evaluation and Results

All figures below are computed on held-out splits of the project's own synthetic datasets (Sections 4.1 and 4.2), using the fixed random seed (42) used throughout the project for reproducibility.

9.1 Document Classification (Classical Models)

Evaluated on the stratified 20% test split (1,200 of 6,000 records) of the document-classification dataset, reporting accuracy and weighted precision, recall, and F1:

Model	Accuracy	Precision	Recall	F1
Logistic Regression	1.0000	1.0000	1.0000	1.0000
Logistic Regression (Scratch)	1.0000	1.0000	1.0000	1.0000
Naive Bayes	1.0000	1.0000	1.0000	1.0000
Naive Bayes (Scratch)	1.0000	1.0000	1.0000	1.0000

Model	Accuracy	Precision	Recall	F1
SVM	1.0000	1.0000	1.0000	1.0000
SVM (Scratch)	1.0000	1.0000	1.0000	1.0000

9.1.1 From-Scratch Training Detail

Because the from-scratch classifiers log their own optimization trace, their training configuration and convergence can be reported directly from the saved result files:

Scratch model	Train / Test	Iterations	Loss (initial → final)
Logistic Regression	4,800 / 1,200	1,000 epochs	0.6931 → 0.1147
Naive Bayes	4,800 / 1,200	closed-form ($\alpha = 1.0$)	n/a (Laplace-smoothed counts)
Linear SVM	4,800 / 1,200	2,000 epochs	1.0000 → 0.0760

All three scratch models used 157 TF-IDF features on this run, a seed of 42, and a learning rate of 0.5 (Logistic Regression) / 0.5 (SVM) with L2 regularization. The strongest learned TF-IDF signals were highly interpretable: features such as “gmail”, “number”, “email”, “phone”, and “amar” (Bengali “my”) pushed the PII class up, while “live”, “learning”, and “bangladesh” pushed the SAFE class up — confirming that the classifiers learned genuinely PII-indicative vocabulary rather than an artifact of the split.

9.2 BiLSTM Sequence Labeling

Trained on 24,000 records and validated on 3,000 held-out records, with a maximum 128-token training window and automatic overlapping-window inference for longer text:

Validation metric	Value
Token accuracy	1.0000
Entity-token precision	1.0000
Entity-token recall	1.0000
Entity-token F1	1.0000
Recommended confidence threshold	0.50
Best epoch (of 5)	5

A separate 3,000-record synthetic held-out test check reproduced the same 1.0000 token accuracy, precision, recall, and F1 at the 0.50 threshold. A manual 152-token long-text inference check correctly detected an email that began after token 150 with confidence 0.9948, confirming that the sliding-window

inference genuinely extends beyond the model's fixed 128-token training window rather than silently truncating.

9.3 Multilingual BERT Token Classification

BERT reports accuracy and micro-averaged precision/recall/F1 computed over non-O token labels only, since the O label dominates raw token counts and would otherwise mask entity-detection errors:

Held-out test metric	Value
Token accuracy	0.999932
Entity-token precision	0.999891
Entity-token recall	0.999837
Entity-token F1	0.999864
Recommended confidence threshold	0.00

Caution when reading these near-perfect scores: the data generator uses a finite bank of templates and entity values, and the train/validation/test split is performed after generation, so closely related sentence patterns can appear across splits. All reported metrics are token-level, not strict whole-span metrics. These results demonstrate that the model has learned the generated distribution extremely well; they do not by themselves demonstrate equivalent accuracy on real emails, clinical text, spelling variants, adversarial formatting, or organizations never seen in the templates.

A methodologically stronger evaluation, identified as future work (Section 12), would add a human-reviewed gold test set, template/source-group-aware splitting performed before augmentation, strict span-level and per-entity-type metrics (e.g. via seqeval or nervaluate), confusion matrices with qualitative error analysis, and evaluation against an independent public PII benchmark.

9.4 Cross-Model Summary

Model family	Task	Best reported metric	Training required?
Pattern / context detector	NER (narrow schema)	n/a (rule-based, deterministic)	No
Classical (LR / NB / SVM × 2)	Document classification	1.0000 weighted F1	Yes (TF-IDF + linear model)

Model family	Task	Best reported metric	Training required?
BiLSTM	NER (18 types, BIO)	1.0000 entity-token F1 (val)	Yes (from random init)
Multilingual BERT	NER (18 types, BIO)	0.9999 entity-token F1 (test)	Yes (fine-tuned from mBERT)

10. Application Layer

10.1 Streamlit Interactive Dashboard

Launched with `python -m streamlit run app.py`, the dashboard provides a responsive, dark-themed privacy interface with:

- all nine model choices with live artifact-availability status;
- per-model confidence threshold and safe-fallback controls;
- built-in English, Bengali, and Banglish example inputs;
- risk-colored result badges and summary metrics;
- confidence progress bars alongside each entity's exact character offsets;
- one-click download of the protected (redacted) text; and
- an explicit notice that all processing happens locally.

Successful BERT/BiLSTM runs return model-only entity detections (patterns are not silently merged in), whereas classical-model runs combine the document-level PII/SAFE prediction with pattern-detected spans, since the classical models cannot localize PII within the text themselves.

11. Testing

The test suite (`python -m pytest`) currently contains 27 tests covering: multi-entity redaction correctness, Bengali text preservation, credit-card Luhn validation, fallback behavior, all nine registry entries, the mathematics of each from-scratch classifier, long-text BiLSTM sliding-window inference, padding-independence of packed sequences, token normalization, BIO label coverage, model-only BERT behavior (no silent pattern merging), and preservation of sensitive tokens through the full pipeline.

12. Limitations and Future Work

12.1 Current Limitations

5. Neural training data is synthetic and drawn from a limited template bank.
6. Records are split into train/validation/test after generation rather than by template family, which can let closely related patterns leak across splits.
7. BIO JSON records do not store original-text character offsets directly.
8. The project uses BIO tagging; BIOES is not implemented.
9. The held-out test set is not a manually reviewed, human-annotated gold set.
10. Pattern-detector coverage is narrower than the full 18-type neural schema.
11. The BiLSTM still maps genuinely unseen rare words to a single shared <UNK> embedding.
12. BiLSTM and BERT validation metrics are token-level and computed on synthetic data rather than a human-reviewed gold set.
13. Classical-model labels are simple weak labels that can reward template memorization rather than genuine PII understanding.
14. The FastAPI service currently exposes the pattern detector only, with no model-selection parameter.
15. Controlled pseudonymization and generative safe-rewriting are not yet implemented — the system detects, classifies, and redacts, but does not rewrite text.

12.2 Highest-Value Next Steps

16. Assign template group IDs and split by group before augmentation, to eliminate template leakage between splits.
17. Add hundreds of structurally different templates and deliberately difficult “hard-negative” safe examples.
18. Build and double-review a smaller, human-annotated gold test set for a defensible accuracy claim.
19. Store original text, token offsets, entity spans, and annotator metadata directly in the dataset format.
20. Add strict span-level and per-entity-type evaluation (e.g., seqeval or nervaluate) alongside the existing token-level metrics.
21. Explore character/sub-word embeddings or a CRF output layer to further improve BiLSTM robustness and BIO-sequence consistency.
22. Extend pattern coverage to passport numbers, account numbers, dates of birth, usernames, and addresses.
23. Add deterministic, format-preserving pseudonymization (e.g., swap a real name for a synthetic one of the same shape).
24. Add a separately evaluated safe-rewriting stage with a mandatory post-generation PII re-scan before output is released.

25. Add API-level model selection and audit metadata without ever logging raw PII.

13. Conclusion

PrivacyGuard demonstrates a complete, end-to-end NLP pipeline for multilingual PII protection: a zero-training rule-based detector as a fast and reliable safety net; three classical machine-learning classifiers implemented both with scikit-learn and entirely from scratch, to make the underlying mathematics explicit; and two neural sequence labelers — a bidirectional LSTM and a fine-tuned multilingual BERT — capable of locating PII at the exact character level across English, Bengali, and Banglish text. A shared, model-agnostic post-processing layer converts any model's raw predictions into confidence-filtered, risk-scored, span-safe redacted output, and the whole system is exposed through both an interactive dashboard and a lightweight API.

On the project's own synthetic benchmarks, every model family reaches near-perfect scores, which is best read as strong evidence that each architecture is implemented correctly and learns its target task well — not yet as a production-readiness claim. The clearest and most valuable next step, explicitly acknowledged throughout this report, is evaluation against real, human-annotated, structurally diverse text, which would convert these encouraging synthetic results into a defensible measure of real-world performance.